LedgerSG Code Review & Audit Report

Comprehensive Critical Analysis of Architecture, Security, and Code Quality

Repository: github.com/nordeim/ledgersg

Date: 2026-05-24

Auditor: Independent Code Review Agent

Classification: Confidential

Table of Contents

Right-click the TOC and select ‘‘Update Field’’ to refresh page numbers after opening.

1. Executive Summary

This report presents the findings of a comprehensive code review and audit of the LedgerSG project, a production-grade double-entry accounting platform purpose-built for Singapore SMBs. The audit covered the entire codebase including the Django 6.0.2 backend, Next.js 16.1.6 frontend, PostgreSQL database schema, and all supporting documentation. The review was conducted against the project's own stated architectural mandates as defined in CLAUDE.md, AGENTS.md, GEMINI.md, README.md, Project_Architecture_Document.md, and API_CLI_Usage_Guide.md.

The audit identified a total of 28 distinct issues across five severity levels: 4 Critical, 5 High, 10 Medium, 5 Low, and 4 Informational. While the project demonstrates strong architectural foundations and consistent adherence to its Service Layer Pattern and SQL-First design philosophy, several critical issues pose immediate risk to data integrity, financial accuracy, and security. These require urgent remediation before the platform can be considered truly production-ready.

The most severe findings include: a dead-code superadmin check that silently bypasses authorization; a missing JWT blacklist that allows revoked tokens continued access; a journal reversal stub that silently skips creating reversal entries for voided invoices; and a hardcoded null UUID in GST calculation that produces incorrect tax computations. Additionally, significant documentation-to-code misalignments were discovered, with endpoint counts, model counts, and version numbers all inconsistent across the various documentation files.

1.1 Severity Distribution

Severity	Count	Description
CRITICAL	4	Data loss, financial corruption, or security breach possible
HIGH	5	Significant functionality broken or security weakness
MEDIUM	10	Degraded functionality, potential race conditions, or code quality issues
LOW	5	Minor code quality, inconsistency, or cosmetic issues
INFO	4	Observations or recommendations for improvement

2. Critical Findings

The following four issues represent the highest-priority risks to the platform. Each one could result in financial data corruption, security breach, or regulatory non-compliance. They must be remediated before any production deployment.

2.1 C-01: Dead Superadmin Authorization Check

ID	Severity	Title
C-01	CRITICAL	is_superadmin field does not exist on AppUser model
Location:	apps/backend/apps/core/permissions.py, apps/backend/common/middleware/tenant_context.py, and 7+ view files	
Description:	The codebase uses getattr(user, 'is_superadmin', False) in multiple locations (permissions.py, tenant_context.py, and various views) to check for superadmin privileges. However, the AppUser model does not define an is_superadmin field. Django's built-in is_superuser field exists, but is_superadmin is never declared. This means every superadmin check always returns False via the default value, making the entire superadmin bypass mechanism dead code. Any user who should have superadmin privileges will be silently denied access, and the fallback to getattr means no error is raised to alert developers of the bug.	
Recommendation:	Replace all getattr(user, 'is_superadmin', False) with user.is_superuser throughout the codebase. Alternatively, if a custom is_superadmin field is desired, add it to the AppUser model and the corresponding SQL schema.	

2.2 C-02: Missing JWT Token Blacklist

ID	Severity	Title
C-02	CRITICAL	JWT token blacklist app missing from INSTALLED_APPS
Location:	apps/backend/config/settings/base.py	
Description:	The logout view calls token.blacklist() to invalidate refresh tokens upon user logout. However, rest_framework_simplejwt.token_blacklist is not included in INSTALLED_APPS. This means the token.blacklist() call either silently fails or raises an ImportError that is swallowed. In either case, refresh tokens are never actually invalidated after logout, allowing any previously issued refresh token to continue generating new access tokens indefinitely. This completely undermines the logout functionality and creates a significant security vulnerability where stolen refresh tokens remain valid even after the user explicitly logs out.	
Recommendation:	Add 'rest_framework_simplejwt.token_blacklist' to INSTALLED_APPS in base.py. Run the corresponding SQL to create the blacklist tables in the database schema.	

ID	Severity	Title
	Verify that token.blacklist() now correctly invalidates tokens by testing the logout flow end-to-end.	

2.3 I-01: Journal Reversal Stub Silently Skips Entries

ID	Severity	Title
I-01	CRITICAL	Document service has duplicate _reverse_journal_entry definitions; the second is a stub
Location:	apps/backend/apps/invoicing/services/document_service.py	
Description:	The document_service.py file contains TWO definitions of the _reverse_journal_entry method within the same class. Python uses the last definition, and the second one is a stub that simply passes without creating any reversal journal entry. This means that when an invoice is voided, the system marks it as VOIDED but never creates the corresponding reversal entries in the General Ledger. The financial statements will therefore continue to reflect revenue and receivables from voided invoices, corrupting P&L and Balance Sheet reports. This is a direct violation of the double-entry accounting principle that every transaction must have balanced entries, including reversals.	
Recommendation:	Remove the stub definition. Implement the proper reversal logic in the first definition (or merge both). The reversal must create debit/credit entries that mirror the original posting, with a reference to the voided document. Wrap in transaction.atomic() along with the document status update.	

2.4 G-01: Hardcoded Null UUID Breaks GST Calculation

ID	Severity	Title
G-01	CRITICAL	GSTCalculateView passes hardcoded null UUID as org_id for tax code lookup
Location:	apps/backend/apps/gst/views.py	
Description:	The GSTCalculateView passes the hardcoded null UUID '00000000-0000-0000-0000-000000000000' as the org_id parameter when calling the GST calculation service. Since tax codes are stored per-organisation and protected by RLS policies that filter by org_id, this null UUID will either fail to match any organisation's tax codes or, worse, match tax codes from an organisation where the null UUID was accidentally used as a seed value. This means GST calculations are fundamentally	

ID	Severity	Title
	broken, producing either zero tax or incorrect tax amounts for all users. In a Singapore context where GST compliance is mandatory and audited by IRAS, this is a critical regulatory risk.	
Recommendation:	Extract the org_id from the authenticated user's context (request.user.default_org_id or from the URL parameter) and pass it correctly to the GST calculation service. Add a validation check that org_id is never null or the null UUID before proceeding with tax code lookups.	

3. High Severity Findings

3.1 C-03: Race Condition in Set Default Organisation

ID	Severity	Title
C-03	HIGH	set_default_org_view clear + set operations not wrapped in transaction.atomic()
Location:	apps/backend/apps/core/views/auth.py (lines 277-282)	
Description:	The set_default_org_view performs a clear of all existing default flags followed by setting a new default organisation. These two operations are not wrapped in a transaction.atomic() block. If the process crashes between the clear and set operations, or if another concurrent request reads the user's organisations between the two operations, the user could end up with no default organisation. This can cause the frontend to display 'No Organisation Selected' unexpectedly and potentially redirect to login, disrupting the user experience.	
Recommendation:	Wrap the clear + set operations in a with transaction.atomic(): block. Additionally, consider using select_for_update() on the UserOrganisation rows to prevent concurrent modifications.	

3.2 C-04: Pending Users Get RLS Permissions

ID	Severity	Title
C-04	HIGH	_get_org_role returns permissions for pending/unaccepted memberships
Location:	apps/backend/common/middleware/tenant_context.py (lines 268-299)	
Description:	The _get_org_role method in TenantContextMiddleware does not filter for accepted_at__isnull=False when looking up UserOrganisation records. This means users who have been invited to an organisation but have not yet accepted the	

ID	Severity	Title
		invitation will have RLS session variables set for that organisation, granting them full access to the organisation's data. This is a significant authorization bypass that violates the intended invitation-acceptance workflow documented in the project architecture.
Recommendation:		Add accepted_at__isnull=False to the UserOrganisation query filter in _get_org_role. This ensures only accepted members can access organisation data through the RLS mechanism.

3.3 I-02: Journal Posting Outside Atomic Block

ID	Severity	Title
I-02	HIGH	approve_document() creates journal entry outside transaction.atomic() block
Location:	apps/backend/apps/invoicing/services/document_service.py	
Description:	In the approve_document() method, the invoice status update is wrapped in transaction.atomic(), but the subsequent call to JournalService.post_invoice() (which creates the journal entries) occurs outside the atomic block. If the journal posting fails (e.g., due to a validation error in the balance check trigger), the invoice remains in APPROVED status but has no corresponding ledger entries. This breaks the fundamental accounting invariant that every approved document must have matching journal entries, leading to incomplete financial statements.	
Recommendation:	Move the JournalService.post_invoice() call inside the transaction.atomic() block so that both the status update and journal entry creation succeed or fail together. This ensures the invariant is maintained: APPROVED status implies ledger entries exist.	

3.4 FE-002: PDF Downloads Completely Broken

ID	Severity	Title
FE-002	HIGH	useInvoicePDF hook reads JWT from localStorage which is never set, causing permanent 401 errors
Location:	apps/web/src/hooks/use-invoices.ts	
Description:	The useInvoicePDF hook attempts to read the JWT access token from localStorage.getItem('access_token'). However, the platform's Zero JWT Exposure mandate means tokens are never stored in localStorage. The access token is stored	

ID	Severity	Title
		in React state/memory and the refresh token is in an HttpOnly cookie. This means PDF downloads always fail with a 401 Unauthorized error because the token is never found. If a developer 'fixes' this by storing the token in localStorage, it would directly violate the project's Zero JWT Exposure security mandate.
Recommendation:		Refactor PDF downloads to go through the server-side API client (lib/server/api-client.ts) which has access to the token without exposing it to the client. Alternatively, create a backend endpoint that generates and returns the PDF using the existing HttpOnly cookie for authentication.

3.5 DB-001: RLS Missing on Core Tables

ID	Severity	Title
DB-001	HIGH	RLS policies missing on core.app_user, core.role, core.user_organisation, and gst.organisation_peppol_settings
Location:	apps/backend/database_schema.sql	
Description:	Four tables in the database schema do not have Row-Level Security policies enabled. While most tables have the standard tenant_isolation RLS policy, these four tables are accessible to any authenticated user regardless of their organisation context. The core.user_organisation table is particularly sensitive because it maps users to organisations and defines their roles; without RLS, a user could potentially query other organisations' membership records. The gst.organisation_peppol_settings table contains InvoiceNow integration credentials that should be isolated per-organisation.	
Recommendation:	Add ALTER TABLE ... ENABLE ROW LEVEL SECURITY and CREATE POLICY tenant_isolation statements for all four tables. For core.app_user and core.role, consider whether cross-tenant visibility is intended and add appropriate policies. For core.user_organisation and gst.organisation_peppol_settings, add org_id-based isolation policies matching the existing pattern.	

4. Medium Severity Findings

The following issues represent degraded functionality, potential race conditions, or code quality concerns that should be addressed in the near term.

4.1 Service Layer Violations

ID	Severity	Title
M-01	MEDIUM	Approximately 5 views contain direct ORM queries or business logic violating the Service Layer Pattern
Location:	JournalEntrySummaryView, DocumentSummaryView, AccountSearchView, ValidateBalanceView, FinancialReportView	
Description:	The project mandates that ALL business logic reside in services/ modules with views serving as thin controllers. However, at least five views contain direct Django ORM queries and business logic that should be extracted into service layer methods. This makes the code harder to test independently and violates the documented architecture. The views include aggregation queries, filtering logic, and response formatting that should be in service classes.	
Recommendation:	Extract all business logic from these views into corresponding service methods. Views should only handle request parsing, service delegation, and response formatting. This maintains testability and adheres to the documented architecture.	

4.2 FE-003: Hardcoded GST Rate

ID	Severity	Title
M-02	MEDIUM	GST rate 9% is hardcoded in gst-engine.ts instead of being fetched from the API
Location:	apps/web/src/lib/gst-engine.ts	
Description:	The frontend GST calculation engine hardcodes the GST rate as 9%. While this matches the current Singapore GST rate, it creates a maintenance risk if the rate changes (as it did from 7% to 8% to 9% in recent years). The backend already has the tax code infrastructure to serve the correct rate dynamically. If the rate changes and the frontend is not redeployed simultaneously, invoice previews will show incorrect GST amounts while the authoritative backend calculation remains correct, creating confusion for users.	
Recommendation:	Fetch the GST rate from the backend API (e.g., via the tax codes endpoint) when the application loads or when an organisation is selected. Cache the rate in TanStack Query and use it in gst-engine.ts. Keep the hardcoded value as a fallback only.	

4.3 FE-005: Invoices Page Uses Mock Data

ID	Severity	Title
M-03	MEDIUM	Invoices page renders mock/hardcoded data instead of fetching from API hooks
Location:	apps/web/src/app/(dashboard)/invoices/page.tsx	
Description:	The invoices listing page currently renders mock data instead of using the existing useInvoices hook to fetch real data from the backend API. This means the invoices page shows fictional data that does not reflect the actual database state. The useInvoices hook exists and is functional, but it is not connected to this page. This creates a disconnect where users see sample invoices that do not exist in their organisation, potentially causing confusion and making the invoicing workflow non-functional.	
Recommendation:	Replace the mock data with a proper useQuery hook call using the existing useInvoices hook. Handle loading, error, and empty states appropriately per the project's UI guidelines.	

4.4 FE-011: FormData Sent via JSON Client

ID	Severity	Title
M-04	MEDIUM	CSV bank statement import sends FormData via JSON API client, which will fail
Location:	apps/web/src/app/(dashboard)/banking/components/import-transactions-form.tsx	
Description:	The CSV import form attempts to send a file upload (FormData) through the standard JSON-based API client. The JSON client sets Content-Type to application/json and serializes the body with JSON.stringify(), which cannot handle FormData objects. File uploads must use Content-Type: multipart/form-data. This means the bank transaction CSV import feature is non-functional, preventing users from reconciling bank statements, which is a core workflow in the banking module.	
Recommendation:	Create a separate file upload function in api-client.ts that uses FormData with Content-Type: multipart/form-data. Update the import form to use this function instead of the standard JSON client.	

4.5 J-03: Race Condition in Journal Entry Numbering

ID	Severity	Title
M-05	MEDIUM	Journal entry numbering lacks select_for_update(), risking duplicate numbers under concurrent access
Location:	apps/backend/apps/journal/services.py	
Description:	The journal entry numbering system uses the document_sequence table but does not call select_for_update() when reading the current sequence value. Under concurrent access (multiple users creating entries simultaneously), two transactions could read the same sequence value and generate duplicate entry numbers. While the database's FOR UPDATE mechanism is documented in the codebase for document numbering, it appears the journal module does not fully implement this pattern. Duplicate entry numbers would violate the document numbering integrity required for audit compliance.	
Recommendation:	Add select_for_update() to the document_sequence query in the journal entry creation path. Ensure the entire sequence increment and document creation is wrapped in transaction.atomic().	

4.6 H-01: SET LOCAL RLS Depends on ATOMIC_REQUESTS

ID	Severity	Title
M-06	MEDIUM	RLS SET LOCAL relies on ATOMIC_REQUESTS=True with no guard or assertion
Location:	apps/backend/common/middleware/tenant_context.py	
Description:	The TenantContextMiddleware uses SET LOCAL to set PostgreSQL session variables for RLS. The SET LOCAL command only persists within the current transaction, and its effectiveness depends entirely on ATOMIC_REQUESTS=True being set in Django settings. If ATOMIC_REQUESTS is ever disabled (which could happen during performance tuning), SET LOCAL would have no effect and RLS would not enforce tenant isolation, causing silent cross-tenant data leakage. There is no assertion or guard to detect this condition.	
Recommendation:	Add an assertion or runtime check in TenantContextMiddleware that verifies ATOMIC_REQUESTS is True. If it is not, raise an ImproperlyConfigured exception or log a critical error. Consider using SET (without LOCAL) with an explicit RESET at the end of the request as a more robust alternative.	

4.7 H-02: Service Settings Drops RLS Middleware

ID	Severity	Title
M-07	MEDIUM	service.py settings file drops TenantContextMiddleware and AuditContextMiddleware
Location:	apps/backend/config/settings/service.py	
Description:	The service.py settings file (used for Celery workers) removes both TenantContextMiddleware and AuditContextMiddleware from the middleware chain. This means Celery tasks run without any RLS context, potentially accessing data across all organisations. While Celery tasks may need to explicitly set their own context, removing the middleware entirely means there is no safety net if a task forgets to set the org context, leading to potential cross-tenant data access.	
Recommendation:	Either retain TenantContextMiddleware in the service settings (with modifications for Celery compatibility) or implement a Celery task mixin/decorator that explicitly sets the RLS context before executing any database operations. Add a safety check that logs a warning if a task accesses the database without RLS context.	

4.8 Remaining Medium Findings

The following additional medium-severity issues were identified during the audit:

- FE-007: isGSTRegistered={true} is hardcoded in the new invoice page instead of being dynamically determined from the organisation's settings, causing incorrect GST behaviour for non-GST registered businesses.
- FE-012: useToast hook has a stale state listener bug where the [state] dependency array causes the listener to not update when state changes, potentially causing toast notifications to be missed.
- FE-010: Duplicate formatCurrency functions exist with different output formats in different parts of the frontend, leading to inconsistent currency display.
- B-01/B-02: Race conditions in payment allocation where concurrent allocations could over-allocate a payment amount due to missing select_for_update() on the payment record.

5. Low & Informational Findings

5.1 Low Severity

- H-03: InvoiceLine inherits from BaseModel instead of TenantModel, inconsistent with InvoiceDocument which uses TenantModel. This creates ambiguity about which base class to use for tenant-scoped models.

- H-04/H-05: OrganisationSetting and ExchangeRate models are missing db_column specifications for 4 fields each, likely causing schema mismatches that could result in runtime query errors.
- H-06: custom_exception_handler has an unreachable ValidationError catch block (dead code path), reducing the effectiveness of error handling for validation errors.
- FE-004: BankAccountsTabProps uses any[] type, violating the TypeScript strict mode mandate of no any types.
- FE-006: Raw HTML input element used instead of Shadcn Input component in one location, violating the UI library discipline mandate.

5.2 Informational

- DB-002/003: Missing indexes on bank_transaction(org_id) and document_line(org_id) could cause slow queries as data grows. Consider adding these for production performance.
- DB-006: System roles with NULL org_id have no tenant isolation, which may be intentional but should be explicitly documented.
- Multiple documentation files reference SECURITY_AUDIT.md and E2E_TEST_EXECUTION_SUMMARY.md, but these files do not exist in the repository.
- Dead symlink: Agent-Browser-howto.md points to /home/pete/.openclaw/workspace/Agent-Browser-howto.md which does not exist.

6. Documentation-Code Alignment Audit

A critical part of this audit was verifying the claims made across the six primary documentation files (CLAUDE.md, AGENTS.md, GEMINI.md, README.md, Project_Architecture_Document.md, API_CLI_Usage_Guide.md) against the actual codebase. Documentation accuracy is especially important for LedgerSG because these files serve as the primary onboarding and reference material for both human developers and AI coding agents. Misaligned documentation can lead to incorrect assumptions, wasted debugging time, and architectural violations.

6.1 Alignment Summary

Claim	Source	Actual	Aligned?
94/87/83 API endpoints	Multiple docs	82 unique resolvable paths	No
30 database tables	PAD, GEMINI.md	29 tables	No
22 Django models	AGENTS.md	27 model classes	No
7 schemas	All docs	7 schemas	Yes

Claim	Source	Actual	Aligned?
Backend v0.3.4	CLAUDE.md, GEMINI.md	v0.3.4 in code	Partial
Backend v0.3.3	README.md, AGENTS.md	Conflicts with v0.3.4	No
28 tables	AGENTS.md	29 actual	No
middleware.ts at apps/web/	README.md	Actually at apps/web/src/middleware.ts	No
RLS on all tables	All docs	Missing on 4 tables	No
100% security score	All docs	CSP report-only, SEC-004/005 open	Overstated
app.current_org_id + app.current_user_id	PAD	Confirmed in code	Yes
9 list views return {results, count}	API Guide	13+ views return format	Understated

6.2 Key Discrepancies

Endpoint Count: Three different numbers (94, 87, 83) are cited across the documentation files, but the actual count of unique resolvable API paths is 82. This suggests the counts were not updated after endpoint refactoring, or were estimated rather than counted. All documentation files should be updated to reflect the accurate count.

Model Count: The documentation claims 22 Django models, but the actual codebase contains 27 model classes (25 in apps/core/models/ plus 2 in apps/peppol/models.py). The discrepancy likely arose from models being added without updating the documentation, and from the fact that many models were consolidated into core/models/ rather than remaining in their respective app directories.

Version Numbers: CLAUDE.md and GEMINI.md list the backend version as v0.3.4, while README.md and AGENTS.md list it as v0.3.3. This inconsistency suggests the version was bumped in some files but not others. A single source of truth for version numbers should be established.

Table Count: AGENTS.md claims 28 tables, while other docs claim 30. The actual count from database_schema.sql is 29 tables. The table count also contradicts itself between documentation files, indicating that no authoritative count was established.

7. Architectural Strengths

Despite the issues identified, the LedgerSG project demonstrates several significant architectural strengths that provide a solid foundation for remediation and continued development.

7.1 Consistent Service Layer Pattern

With the exception of approximately 5 views, the Service Layer Pattern is consistently followed across all backend modules. Views act as thin controllers that deserialize input, delegate to service methods, and serialize output. Business logic is properly encapsulated in service classes, making the code testable and maintainable. This is a commendable architectural discipline that should be preserved and extended to the few remaining violators.

7.2 SQL-First Design with Unmanaged Models

The SQL-First approach with managed = False on all Django models is consistently implemented. The database_schema.sql file serves as the true source of truth, and the codebase correctly avoids Django migrations. The NUMERIC(10,4) precision for all monetary values is enforced at the database level, and the money() utility is used consistently for currency operations with no instances of float arithmetic found. The journal.validate_balance() trigger is correctly implemented as a deferred constraint, ensuring double-entry integrity at the database level.

7.3 Defense-in-Depth Authentication

The three-layer authentication architecture (AuthProvider, DashboardLayout Guard, Backend JWT) is well-designed and properly implemented. The CORSJWTAuthentication class correctly handles the CORS preflight problem by skipping OPTIONS requests. The JWT strategy with 15-minute access tokens and 7-day refresh tokens in HttpOnly cookies is appropriate for the security requirements. The implementation of CSP headers (even in report-only mode) and rate limiting demonstrates security consciousness.

7.4 Comprehensive RLS Implementation

Row-Level Security is enforced on the vast majority of tables (25 of 29), which is a strong foundation. The TenantContextMiddleware correctly sets PostgreSQL session variables using SET LOCAL within the request transaction. The RLS policies follow a consistent pattern (tenant_isolation with org_id = core.current_org_id()), making them easy to understand and audit. The few missing RLS policies identified in this audit can be added straightforwardly.

7.5 Test Infrastructure

The project has a substantial test infrastructure with 700+ tests across frontend and backend. The TDD methodology (RED-GREEN-REFACTOR) is documented and followed for new features. The pytest configuration with --reuse-db --no-migrations correctly handles the unmanaged model constraint. The frontend test suite uses Vitest with React Testing Library and correctly uses userEvent.setup() for Radix UI component testing.

8. Prioritized Remediation Plan

The following remediation plan is organized into three phases based on urgency and dependency. Each phase should be completed and verified before moving to the next.

8.1 Phase 1: Critical Fixes (Immediate, 1-3 Days)

These fixes address data integrity, security, and financial accuracy issues that could cause immediate harm in production.

ID	Issue	Fix	Est. Effort
C-01	is_superadmin dead code	Replace with is_superuser	2 hours
C-02	Missing JWT blacklist	Add to INSTALLED_APPS + SQL	4 hours
I-01	Journal reversal stub	Implement proper reversal logic	1 day
G-01	Hardcoded null UUID in GST	Extract org_id from request context	2 hours
I-02	Journal posting outside atomic	Move into transaction.atomic() block	1 hour

8.2 Phase 2: High Priority Fixes (1 Week)

These fixes address authorization bypasses, broken functionality, and significant security gaps.

ID	Issue	Fix	Est. Effort
C-03	Race condition set_default_org	Wrap in transaction.atomic()	1 hour
C-04	Pending users get RLS permissions	Add accepted_at__isnull=False filter	1 hour
FE-002	PDF downloads broken	Refactor to server-side API client	4 hours
DB-001	RLS missing on 4 tables	Add RLS policies to database_schema.sql	3 hours

8.3 Phase 3: Documentation & Code Quality (2 Weeks)

These fixes address documentation misalignments, code quality issues, and architectural consistency.

ID	Issue	Fix	Est. Effort
DOC-01	Endpoint count inconsistent	Count actual endpoints, update all docs	3 hours
DOC-02	Model/table counts wrong	Count actual models/tables, update all docs	2 hours
DOC-03	Version numbers inconsistent	Unify to single version across all docs	1 hour
M-01	5 views violate service layer	Extract logic to service methods	1 day
M-02	Hardcoded GST rate	Fetch from API dynamically	4 hours
M-03	Invoices page mock data	Connect to useInvoices hook	2 hours
M-04	FormData sent via JSON client	Create multipart upload function	2 hours

9. Strategic Recommendations

9.1 Single Source of Truth for Metrics

The project has six documentation files that all contain overlapping metrics (test counts, endpoint counts, version numbers, table counts). These frequently fall out of sync, as demonstrated by this audit. The recommendation is to create a single metrics.json file (or a generated section) that serves as the authoritative source, with all other documentation files referencing it rather than duplicating the values. This could be auto-generated as part of the CI/CD pipeline by running scripts that count actual endpoints, models, tests, and tables.

9.2 Add Pre-commit Hooks for Documentation Consistency

Implement pre-commit hooks or CI checks that verify documentation claims against the actual codebase. For example, a script that counts URL patterns and compares them to the documented endpoint count, or that verifies the model count matches the documentation. This would prevent documentation drift and ensure that developers are working from accurate information, which is especially important for a project that relies heavily on AI coding agents that reference these documentation files.

9.3 Implement End-to-End Contract Testing

While the project has API contract tests, they should be expanded to cover all list endpoints and verify the {results, count} format consistently. Consider implementing consumer-driven contract testing (e.g., using Pact) to formally define the contract between the frontend and backend,

preventing the kind of mismatch that broke the Banking module. This is particularly important given that the frontend and backend are in separate codebases with independent deployment cycles.

9.4 Security Posture Enhancement

While the project claims a 100% security score, this audit found several security-relevant issues (missing RLS, JWT blacklist, pending user access). The recommendation is to qualify the security score claim by noting that CSP is in report-only mode and SEC-004/005 remain open. Additionally, consider implementing automated security testing in the CI pipeline that checks for RLS policies on all tables, validates JWT blacklist functionality, and verifies that all list endpoints return the expected paginated format.

9.5 Database Migration Strategy

The SQL-First approach is a strength, but the lack of a formal migration strategy creates risks. When schema changes are needed, developers must manually write SQL patches and update Django models. Consider creating a structured migration directory (separate from Django's migration system) that tracks SQL patches in order, similar to Flyway or Liquibase. Each patch should include both the up and down migration, along with a description of the change and its impact on Django models.

10. Conclusion

The LedgerSG project demonstrates strong architectural foundations with its SQL-First design, Service Layer Pattern, and defense-in-depth security approach. The core design decisions are sound and well-documented, providing a solid framework for building a compliant accounting platform for Singapore SMBs.

However, this audit has identified 28 issues ranging from critical data integrity bugs to documentation misalignments. The four critical findings (dead superadmin check, missing JWT blacklist, journal reversal stub, and hardcoded null UUID in GST calculation) represent immediate risks that must be addressed before the platform can be safely deployed to production. The high-severity findings (race conditions, broken PDF downloads, missing RLS policies) further underscore the need for thorough remediation.

The most concerning pattern revealed by this audit is not any single bug, but rather the gap between the documentation's claims of production readiness and the actual state of the code. Documentation stating 100% security scores, specific test counts, and precise endpoint numbers that do not match reality can mislead both human developers and AI agents into making incorrect assumptions. The remediation plan provided in Section 8 prioritizes fixing these discrepancies alongside the code-level issues, ensuring that the project's documentation accurately reflects its implementation.

With the recommended fixes applied, particularly the Phase 1 critical fixes, the LedgerSG platform would be well-positioned for production deployment. The underlying architecture is robust, and the issues identified are largely implementation gaps rather than fundamental design flaws.